

# Applications of Parallel Programming

Chris Kauffman

*Last Updated:*

*Tue May 5 01:49:38 PM EDT 2026*

# Logistics

## Assignment grading commenced

Likely to finish by end of week

## Endgame

| Date       | Event                         |
|------------|-------------------------------|
| Wed 29-Apr | A4 Due                        |
| Thu 30-Apr | ME4, Applications             |
| Tue 05-May | Applications                  |
| Thu 07-May | Finale                        |
| Thu 14-May | Final Exam<br>10:30am-12:30pm |

## Course Experiences Now Open

e.g. Rate your Professor

- ▶ <https://www.courseexp.umd.edu/>
- ▶ **If response rate reaches 80% for every section...**
- ▶ **by Sat 09-May-2026 11:59pm...**
- ▶ **I will reveal a Final Exam Question**
- ▶ No answers but public discussion welcome

## Exercise: Revisiting A3's Matrix Load Comparison

```
int lu_load_mpi1(char *fname, float **Ap, size_t *dimp){
    ...;
    size_t row_count = dim / proc_count;
    size_t row_start = proc_rank * row_count*dim*sizeof(float);
    fseek(infile, row_start, SEEK_CUR);
    float *A = malloc(sizeof(float)*row_count*dim);
    elems_read = fread(A, sizeof(float), row_count*dim, infile);
    ...;
}

int lu_load_mpi2(char *fname, float **Ap, size_t *dimp){
    ...;
    float *Aall = NULL;
    if(proc_rank == 0){
        Aall = malloc(sizeof(float)*dim*dim);
        size_t rowbytes = sizeof(float)*dim;
        for(int p=0; p<proc_count; p++){
            size_t file_offset = sizeof(size_t) + p*rowbytes;
            size_t matr_offset = p*dim*row_count;
            for(size_t r=0; r<row_count; r++){
                fseek(infile, file_offset+rowbytes*proc_count*r,
                    SEEK_SET);
                fread(Aall+matr_offset+r*dim, sizeof(float),
                    dim, infile);
            }
        }
    }
    float *A = malloc(sizeof(float)*row_count*dim);
    MPI_Scatter(Aall, row_count*dim, MPI_FLOAT,
        A, row_count*dim, MPI_FLOAT,
        0, MPI_COMM_WORLD);
    ...;
}
```

Take a moment and recall the **differences** between these two input approaches which was more efficient

## Answers: Revisiting A3's I/O Approach

- ▶ `lu_load_mpi1()` had each MPI process reading files from different offsets to load its portion of the data; each proc loaded a Contiguous Rows for a Block Row Partition
- ▶ `lu_load_mpi2()` had Root Proc loading the entire matrix and then scattering it; loaded in a specific arrangement to allow for a Cyclic Row Partition
- ▶ Loading was faster for `lu_load_mpi1()` as **loading could happen in parallel**

This pattern is common: all processors need a portion of an input file (or write a portion of an output file) so...

- ▶ MPI provides some facilities for this AND
- ▶ Hardware can provide a Parallel File System that speeds operations

# MPI Library for Parallel I/O

- ▶ Basic operations are similar to C library (`fread()/fwrite()/fseek()`) or POSIX (`read()/write()/lseek()`)
- ▶ Calls are either **Individual or Collective** with collective calls asking the backing file system / hardware to batch operations
- ▶ Also provides **File Views** that alter locations of reading per Process

# Basic and Individual Calls

```
// mirror the POSIX standards
int MPI_File_open(MPI_Comm comm, char *filename, int amode, MPI_Info info,
                  MPI_File *fh);
int MPI_File_close(MPI_File *fh);

////////////////////////////////////
// NON-COLLECTIVE I/O operations so WILL benefit from any underlying
// efficiencies of tuned I/O system

// like read()/write() or fread()/fwrite()
int MPI_File_read(MPI_File fh,
                  void *buf, int count, MPI_Datatype datatype,
                  MPI_Status *status);
int MPI_File_write(MPI_File fh,
                  void *buf, int count, MPI_Datatype datatype,
                  MPI_Status *status)

// like pread()/pwrite() which allows writing at an offset
int MPI_File_read_at(MPI_File fh, MPI_Offset offset,
                    void *buf, int count, MPI_Datatype datatype,
                    MPI_Status *status)
int MPI_File_write_at(MPI_File fh, MPI_Offset offset,
                    void *buf, int count, MPI_Datatype datatype,
                    MPI_Status *status);
```

# Collective Calls

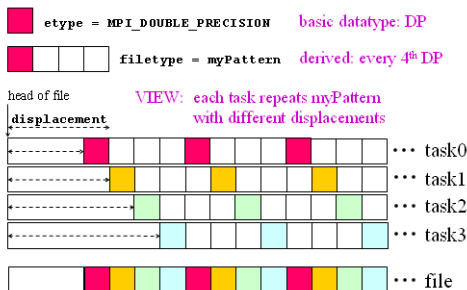
```
/////////////////////////////////////////////////////////////////
// COLLECTIVE I/O operations so WILL benefit from any underlying
// efficiencies of tuned I/O system. These calls MUST be made by all
// PEs in the communicator and allow the file system under the hood to
// make broader efficiency decisions such as delaying actual writes to
// disk until all processors have "written" their data to the
// filesystem temporary space then committing the data in whole to
// permanent storage over the network.

// like read()/write() or fread()/fwrite()
int MPI_File_read_all(MPI_File fh,
                      void *buf, int count, MPI_Datatype datatype,
                      MPI_Status *status);
int MPI_File_write_all(MPI_File fh,
                       void *buf, int count, MPI_Datatype datatype,
                       MPI_Status *status);

// like pread()/pwrite() which allows writing at an offset
int MPI_File_read_at_all(MPI_File fh, MPI_Offset offset,
                          void *buf, int count, MPI_Datatype datatype,
                          MPI_Status *status);
int MPI_File_write_at_all(MPI_File fh, MPI_Offset offset,
                           void *buf, int count, MPI_Datatype datatype,
                           MPI_Status *status);
```

# File Views

```
int MPI_File_set_view(MPI_File fh, MPI_Offset disp, MPI_Datatype etype,  
                     MPI_Datatype filetype, char *datarep, MPI_Info info);  
// - fh: file handle (handle)  
// - disp: displacement (nonnegative integer)  
// - etype: elementary datatype (handle)  
// - filetype: filetype (handle)  
// - datarep: data representation (string)  
// - info: info object (handle)
```

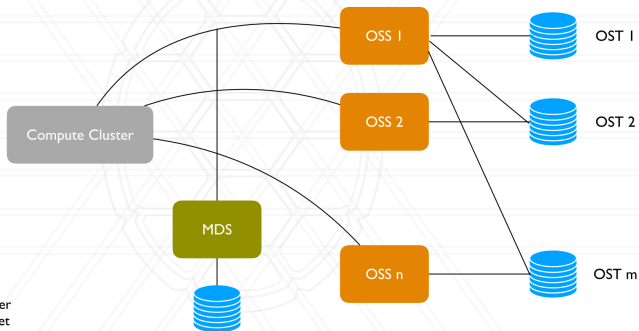


Source: Cornell Virtual Workshop on MPI



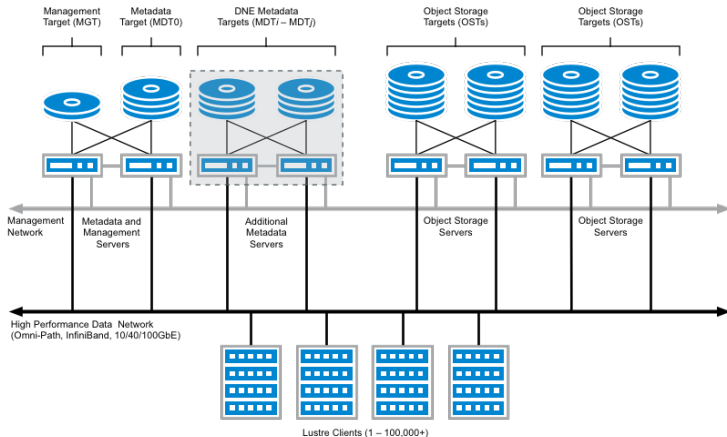
# File Systems for HPC Operations

- ▶ Several File Systems deployed in HPC systems focused on
  - ▶ High performance operations
  - ▶ Parallel I/O
- ▶ Concept: Provide multiple disks that can server the same files or parts of the same file independently to multiple nodes



# Lustre File System

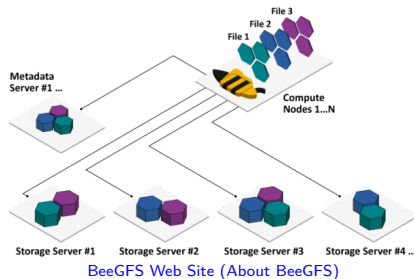
- ▶ Lustre: “L”inux Cl“ustre” file system
- ▶ Open source and oft-deployed to clusters
- ▶ 20+ year old software system for parallel I/O
- ▶



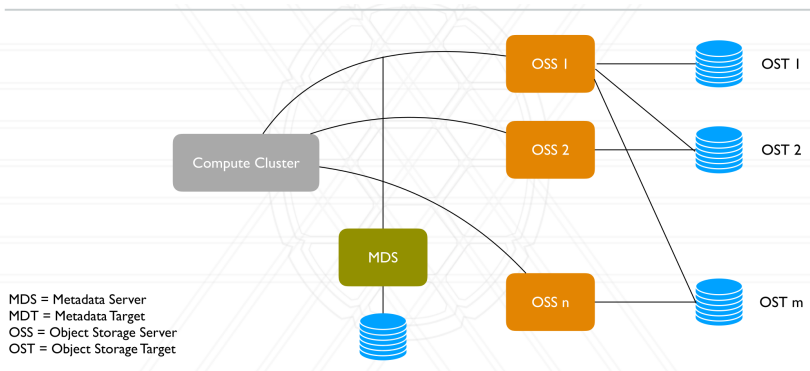
Source: [Learn Lustre / Docs](#)

## Other Options

- ▶ Lustre is not the only game in town
- ▶ BeeGFS has both community and commercial support, used on Zaratán
- ▶ GPFS/Spectrum Scale (commercial through IBM)



# Conceptual Arrangement for Parallel I/O



## I/O on Zaratán

- ▶ Home Directories: Network File System (NFS), reasonable performance (but not High Performance), backed up nightly, limited space (10GB),  
Admins: *Don't run jobs with files in your home directory...*
- ▶ Scratch: BeeGFS, symbolic scratch links in home directory to /scratch, High Performance / Parallel File system, available on all nodes, NOT backed up, larger space allocation (1TB) shared by all project members, meant to load job files and results files
- ▶ SHELL: Medium term storage, Auristor File system, not backed up, not available on compute nodes
- ▶ /tmp directory: available on compute nodes, very high performance BUT all data is deleted when a job completes

Typical use cases would be:

- ▶ Download data to Scratch space prior to a job run
- ▶ Jobs load data from Scratch during a run
- ▶ Write checkpoint files to Scratch and final results
- ▶ After a job is done, move overall results to Home directory or

# Takehome

- ▶ Optimizing I/O operations is yet another angle of attack for HPC
- ▶ MPI provides operations that may be backed by hardware / network
- ▶ HPC systems usually have several kinds of storage and arrangement of disks to enable parallel I/O operations